```
---------------Mandatory Information to fill----------
```

Group ID: 153

Group Members Name with Student ID:

Rajeev Kumar Jha- 2024aa05194 ,Contribution - 100% Ashvini Satish Bhosale - 2024aa05732 ,Contribution - 100% Naresh Kumar - 2024ab05178 ,Contribution - 100% Dhruvi Nileshbhai Makati - 2024aa05575 ,Contribution - 100%

```
------------------Write your remarks (if any) that you want should get consider at the time of evaluation--------------
```

Remarks: ##Add here

Objective:

The goal of this assignment is to model the ICU treatment process using Reinforcement Learning, specifically the Actor-Critic method. The agent should learn an optimal intervention policy from historical ICU data. Each patient's ICU stay is treated as an episode consisting of time-stamped clinical observations and treatments. Your tasks:

1. Model the ICU treatment process as a Reinforcement Learning (RL) environment.
2. Train an Actor-Critic agent to suggest medical interventions based on the patient's current state (vitals and demographics).

Dataset:

Use the dataset provided in the following link:

https://drive.google.com/file/d/1UPsOhUvyrsrC59ilXsvHwGZhzm7Yk01w/view?usp=sharing

Features:

• *Vitals*: mean_bp, spo2, resp_rate

• *Demographics*: age, gender

• *Action*: Medical intervention (e.g., "Vancomycin", "NaCl 0.9%", or NO_ACTION)

• *Identifiers*: timestamp, subject_id, hadm_id, icustay_id

State Space :

Each state vector consists of: mean_bp (Mean Blood Pressure) , spo2 (Oxygen Saturation), resp_rate (Respiratory Rate), age, One-hot encoded gender

Action Space :

• The agent selects one discrete action from 99 possible medical interventions (e.g., Vancomycin, Fentanyl, PO Intake, etc.

• You should integer encode or one-hot encode these interventions.

Reward Function:

At each time step, the agent receives a reward based on how close the patient's vitals are to clinically normal ranges. The reward encourages the agent to take actions that stabilize the patient's vital signs:

$$\text{Reward}_t = -\left((MBP_t - 90)^2 + (SpO2_t - 98)^2 + (RR_t - 16)^2\right)$$

Explanation:

• MBP (mean_bp): Target = 90 mmHg

• SpO_2 (spo2): Target = 98%

• RR (resp_rate): Target = 16 breaths/min

Each term penalizes the squared deviation from the healthy target. The smaller the difference, the higher (less negative) the reward.

Example:

Suppose at time t, the vitals are:

• MBP = 88

• SpO_2 = 97

• RR = 20

Then the reward is:

$$\text{Reward}_t = -\left[(88 - 90)^2 + (97 - 98)^2 + (20 - 16)^2\right] = -(4 + 1 + 16) = -21$$

A lower (more negative) reward indicates worse vitals, guiding the agent to learn actions that minimize this penalty.

📍 Episode Termination

An episode ends when the ICU stay ends. To define this:

1. **Group the data** by `subject_id`, `hadm_id`, and `icustay_id`
   → Each group represents one ICU stay = one episode.
2. **Sort each group** by `timestamp`
   → Ensures the time progression is correct.
3. **For each time step** in a group (i.e., each row):
   → Check if it is the **last row** in that group.
   - If **yes**, then mark `done = True` (end of episode)
   - If **no**, then mark `done = False` (continue episode)

Requirements and Deliverables:

Implement the Sepsis Treatment Optimization Problem for the given above scenario for the below mentioned RL method.

Initialize constants

In [ ]:
```
# Constants
```

Load Dataset (0.5 Mark)

In [1]:
```
from google.colab import files
import pandas as pd

# Upload file (this will open a file picker)
uploaded = files.upload()

# Get the uploaded filename
filename = list(uploaded.keys())[0]

# Load dataset
df = pd.read_csv(filename)

print("Dataset shape:", df.shape)
print(df.head())
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving Sepsis_datset.csv to Sepsis_datset.csv
Dataset shape: (9334, 10)
   subject_id  hadm_id  icustay_id      timestamp  mean_bp  spo2  resp_rate  \
0       40124   126179      279554  2/4/2130 4:42      101   100         15
1       40124   126179      279554  2/4/2130 5:00       78   100         15
2       40124   126179      279554  2/4/2130 5:01       78   100         15
3       40124   126179      279554  2/4/2130 5:03       78    99         15
4       40124   126179      279554  2/4/2130 6:00       63   100         17

   age gender     action
0   67      F  NO_ACTION
1   67      F  NO_ACTION
2   67      F  NO_ACTION
3   67      F  NO_ACTION
4   67      F  NO_ACTION
```

In [2]:
```
# Convert to datetime
df['timestamp'] = pd.to_datetime(df['timestamp'])

# Sort within each ICU stay
df = df.sort_values(by=['subject_id','hadm_id','icustay_id','timestamp']).reset_ind

print(df[['subject_id','hadm_id','icustay_id','timestamp']].head())
```

```
   subject_id  hadm_id  icustay_id           timestamp
0       40124   126179      279554 2130-02-04 04:42:00
1       40124   126179      279554 2130-02-04 05:00:00
2       40124   126179      279554 2130-02-04 05:01:00
3       40124   126179      279554 2130-02-04 05:03:00
4       40124   126179      279554 2130-02-04 06:00:00
```

In [3]:
```
# Encode gender
df['gender'] = df['gender'].astype(str)
df['gender_enc'], gender_mapping = pd.factorize(df['gender'])

# Encode action
df['action_enc'], action_mapping = pd.factorize(df['action'])

print("Gender mapping:", dict(enumerate(gender_mapping)))
print("First 5 action mappings:", dict(enumerate(action_mapping[:5])))
```

```
Gender mapping: {0: 'F', 1: 'M'}
First 5 action mappings: {0: 'NO_ACTION', 1: 'NaCl 0.9%', 2: 'Haloperidol (Haldol)',
3: 'Piggyback', 4: 'Fentanyl'}
```

Design a SepsisTreatmentEnv Environment (0.5 Mark)

In [4]:
```
# Code for environment creation
#-----write your code below this line---------

import numpy as np

class SepsisTreatmentEnv:
    def __init__(self, df):
        """
        df : preprocessed DataFrame containing
```

```
            ['subject_id','hadm_id','icustay_id','timestamp',
             'mean_bp','spo2','resp_rate','age','gender_enc','action_enc']
        """

        self.df = df

        # a) State features
        self.state_cols = ['mean_bp','spo2','resp_rate','age','gender_enc']

        # b) Action space (99 unique actions including 'NO_ACTION')
        self.actions = sorted(df['action'].unique())
        self.n_actions = len(self.actions)

        # Map action string <-> int
        self.action2id = {a:i for i,a in enumerate(self.actions)}
        self.id2action = {i:a for a,i in self.action2id.items()}

        # c) Episode structure: one ICU stay = one episode
        self.episodes = []
        groups = df.groupby(['subject_id','hadm_id','icustay_id']).groups
        for _, idxs in groups.items():
            idxs = sorted(list(idxs))
            self.episodes.append(idxs)

        # Initialize state
        self.current_episode = None
        self.current_step = None

    def reset(self):
        """ Start a new ICU stay (episode). """
        self.current_episode = self.episodes[np.random.randint(len(self.episodes))]
        self.current_step = 0
        row = self.df.loc[self.current_episode[self.current_step]]
        return row[self.state_cols].values.astype(np.float32)

    def step(self, action):
        """
        Move to the next timestep in the same ICU stay.
        Note: We are using offline data, so next state comes from dataset,
              not from chosen action.
        """
        # current row
        row = self.df.loc[self.current_episode[self.current_step]]

        # compute reward from vitals
        mbp, spo2, rr = row['mean_bp'], row['spo2'], row['resp_rate']
        reward = -((mbp - 90)**2 + (spo2 - 98)**2 + (rr - 16)**2)

        # check if this is the last step in the episode
        done = (self.current_step == len(self.current_episode)-1)

        # advance step
        if not done:
            self.current_step += 1
            next_row = self.df.loc[self.current_episode[self.current_step]]
            next_state = next_row[self.state_cols].values.astype(np.float32)
```

```
        else:
            next_state = None

        return next_state, reward, done, {"true_action": row['action_enc']}

# ----------------- Quick test -----------------
env = SepsisTreatmentEnv(df)

state = env.reset()
print("Initial state vector:", state)

next_state, reward, done, info = env.step(0)
print("Next state:", next_state)
print("Reward:", reward)
print("Done:", done)
print("Info (true action in dataset):", info)
```

```
Initial state vector: [ 65. 100.  11.  66.   1.]
Next state: [ 65. 100.  30.  66.   1.]
Reward: -654
Done: False
Info (true action in dataset): {'true_action': np.int64(1)}
```

Implement the Reward Function (1 Mark)

In [5]:
```
# Code for reward function
#-----write your code below this line-------

def compute_reward(mean_bp, spo2, resp_rate):
    """
    Compute reward based on patient vitals.
    Healthy targets:
        mean_bp = 90
        spo2    = 98
        resp_rate = 16
    Reward is negative squared error from these targets.
    """
    reward = -((mean_bp - 90)**2 + (spo2 - 98)**2 + (resp_rate - 16)**2)
    return reward

# ----------------- Quick test -----------------
print("Test case 1: MBP=90, SpO2=98, RR=16 →", compute_reward(90,98,16))
print("Test case 2: MBP=88, SpO2=97, RR=20 →", compute_reward(88,97,20))
print("Test case 3: MBP=110, SpO2=92, RR=10 →", compute_reward(110,92,10))
```

```
Test case 1: MBP=90, SpO2=98, RR=16 → 0
Test case 2: MBP=88, SpO2=97, RR=20 → -21
Test case 3: MBP=110, SpO2=92, RR=10 → -472
```

Design and train Actor-Critic Algorithm (2.5 Mark)

In [8]:
```
# Code for training
#-----write your code below this line------
import torch
import torch.nn as nn
```

```
import torch.optim as optim
import numpy as np
from tqdm import trange

# ----------------- Prepare dataset into transitions -----------------
feature_cols = ['mean_bp','spo2','resp_rate','age','gender_enc']

transitions = []  # list of (s, a, r, s_next, done)

for episode in df.groupby(['subject_id','hadm_id','icustay_id']):
    rows = episode[1].sort_values('timestamp')
    for i in range(len(rows)-1):
        s = rows.iloc[i][feature_cols].values.astype(np.float32)
        a = rows.iloc[i]['action_enc']
        r = -((rows.iloc[i]['mean_bp']-90)**2 +
              (rows.iloc[i]['spo2']-98)**2 +
              (rows.iloc[i]['resp_rate']-16)**2)
        s_next = rows.iloc[i+1][feature_cols].values.astype(np.float32)
        done = 0
        transitions.append((s,a,r,s_next,done))
    # last row of ICU stay
    s = rows.iloc[-1][feature_cols].values.astype(np.float32)
    a = rows.iloc[-1]['action_enc']
    r = -((rows.iloc[-1]['mean_bp']-90)**2 +
          (rows.iloc[-1]['spo2']-98)**2 +
          (rows.iloc[-1]['resp_rate']-16)**2)
    done = 1
    transitions.append((s,a,r,None,done))

print("Total transitions:", len(transitions))

# ----------------- Neural networks -----------------
state_dim = len(feature_cols)
action_dim = df['action_enc'].nunique()

class Actor(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        self.fc2 = nn.Linear(128, 128)
        self.out = nn.Linear(128, action_dim)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return torch.softmax(self.out(x), dim=-1)

class Critic(nn.Module):
    def __init__(self, state_dim):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, 128)
        self.fc2 = nn.Linear(128, 128)
        self.out = nn.Linear(128, 1)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.out(x)
```

```
actor = Actor(state_dim, action_dim)
critic = Critic(state_dim)

actor_opt = optim.Adam(actor.parameters(), lr=3e-4)
critic_opt = optim.Adam(critic.parameters(), lr=3e-4)

# ----------------- Training Loop -----------------
gamma = 0.99
batch_size = 256
n_epochs = 20  # train for more epochs

all_rewards = []
actor_losses = []
critic_losses = []

for epoch in range(n_epochs):
    np.random.shuffle(transitions)
    total_reward = 0
    epoch_actor_loss = 0
    epoch_critic_loss = 0
    n_batches = 0

    for i in range(0, len(transitions), batch_size):
        batch = transitions[i:i+batch_size]
        states = torch.tensor([b[0] for b in batch], dtype=torch.float32)
        actions = torch.tensor([b[1] for b in batch], dtype=torch.long)
        rewards = torch.tensor([b[2] for b in batch], dtype=torch.float32)
        next_states = [b[3] for b in batch]
        non_final_mask = torch.tensor([s is not None for s in next_states], dtype=t
        non_final_next = torch.tensor([s for s in next_states if s is not None], dt
        dones = torch.tensor([b[4] for b in batch], dtype=torch.float32)

        # Critic loss (TD error)
        values = critic(states).squeeze()
        next_values = torch.zeros_like(values)
        if non_final_next.shape[0] > 0:
            next_values[non_final_mask] = critic(non_final_next).squeeze().detach()
        targets = rewards + gamma * next_values * (1 - dones)
        critic_loss = nn.MSELoss()(values, targets)

        critic_opt.zero_grad()
        critic_loss.backward()
        critic_opt.step()

        # Actor loss (policy gradient with advantage)
        probs = actor(states)
        dist = torch.distributions.Categorical(probs)
        log_probs = dist.log_prob(actions)
        advantages = (targets - values).detach()
        actor_loss = -(log_probs * advantages).mean()

        actor_opt.zero_grad()
        actor_loss.backward()
        actor_opt.step()
```

```
        total_reward += rewards.sum().item()
        epoch_actor_loss += actor_loss.item()
        epoch_critic_loss += critic_loss.item()
        n_batches += 1

    avg_reward = total_reward / len(transitions)
    all_rewards.append(avg_reward)
    actor_losses.append(epoch_actor_loss / n_batches)
    critic_losses.append(epoch_critic_loss / n_batches)

    print(f"Epoch {epoch+1}/{n_epochs} | Avg reward: {avg_reward:.2f} | "
          f"Actor loss: {actor_losses[-1]:.4f} | Critic loss: {critic_losses[-1]:.4
```

```
Total transitions: 9334
Epoch 1/20 | Avg reward: -565.39 | Actor loss: -8783.1481 | Critic loss: 697000.3361
Epoch 2/20 | Avg reward: -565.39 | Actor loss: -8983.7885 | Critic loss: 696740.2500
Epoch 3/20 | Avg reward: -565.39 | Actor loss: -8915.3362 | Critic loss: 692467.2559
Epoch 4/20 | Avg reward: -565.39 | Actor loss: -8796.0886 | Critic loss: 690156.3573
Epoch 5/20 | Avg reward: -565.39 | Actor loss: -8607.6137 | Critic loss: 692551.3775
Epoch 6/20 | Avg reward: -565.39 | Actor loss: -8355.7429 | Critic loss: 725371.5752
Epoch 7/20 | Avg reward: -565.39 | Actor loss: -7921.9626 | Critic loss: 791259.1883
Epoch 8/20 | Avg reward: -565.39 | Actor loss: -7420.1546 | Critic loss: 931358.3750
Epoch 9/20 | Avg reward: -565.39 | Actor loss: -6719.1120 | Critic loss: 1254525.126
7
Epoch 10/20 | Avg reward: -565.39 | Actor loss: -5940.0347 | Critic loss: 1733906.90
88
Epoch 11/20 | Avg reward: -565.39 | Actor loss: -4995.7419 | Critic loss: 2500800.59
80
Epoch 12/20 | Avg reward: -565.39 | Actor loss: -4096.7091 | Critic loss: 3433206.14
02
Epoch 13/20 | Avg reward: -565.39 | Actor loss: -3360.8658 | Critic loss: 4392460.41
22
Epoch 14/20 | Avg reward: -565.39 | Actor loss: -2435.3895 | Critic loss: 5588885.63
85
Epoch 15/20 | Avg reward: -565.39 | Actor loss: -2093.8468 | Critic loss: 6151889.65
20
Epoch 16/20 | Avg reward: -565.39 | Actor loss: -1529.6736 | Critic loss: 7122243.50
00
Epoch 17/20 | Avg reward: -565.39 | Actor loss: -1484.6591 | Critic loss: 6972761.37
50
Epoch 18/20 | Avg reward: -565.39 | Actor loss: -1274.8794 | Critic loss: 7338781.53
04
Epoch 19/20 | Avg reward: -565.39 | Actor loss: -1095.6262 | Critic loss: 7609035.87
16
Epoch 20/20 | Avg reward: -565.39 | Actor loss: -1076.8681 | Critic loss: 7565670.85
14
```

Plot the graph for Average Reward (1 Mark)

In [9]:

```
# Code for plotting the average reward
#-----write your code below this line------
import matplotlib.pyplot as plt

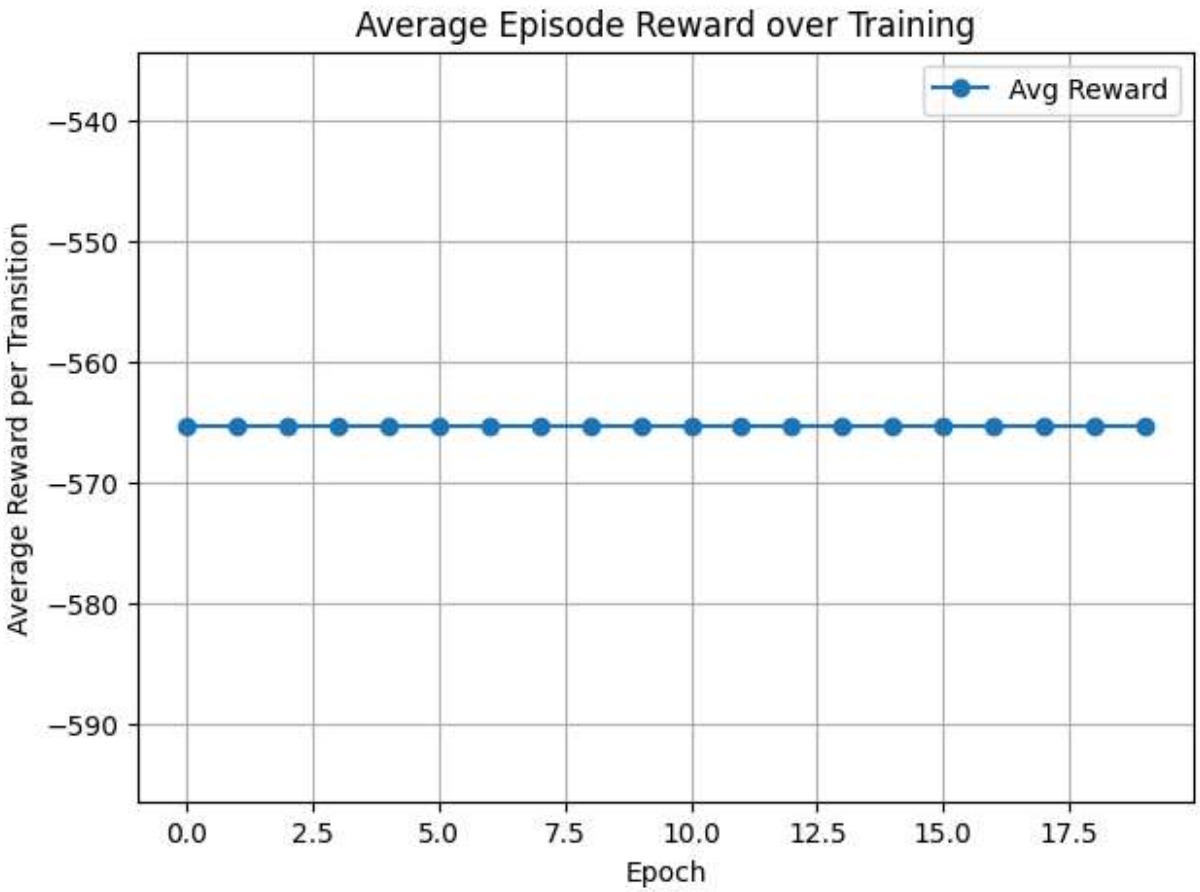
# ----------------- Plot Average Episode Reward -----------------
plt.figure(figsize=(7,5))
```

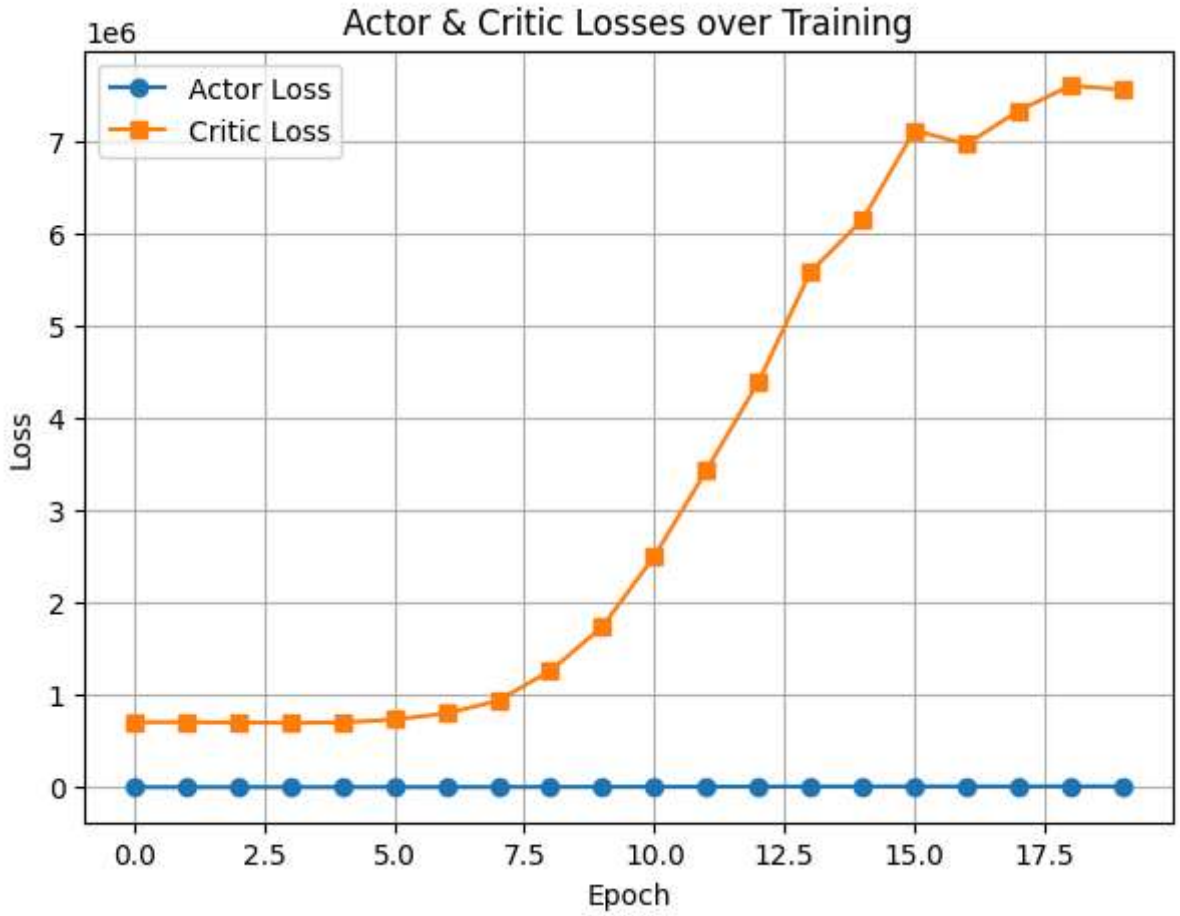
```
plt.plot(all_rewards, marker='o', label="Avg Reward")
plt.xlabel("Epoch")
plt.ylabel("Average Reward per Transition")
plt.title("Average Episode Reward over Training")
plt.grid(True)
plt.legend()
plt.show()

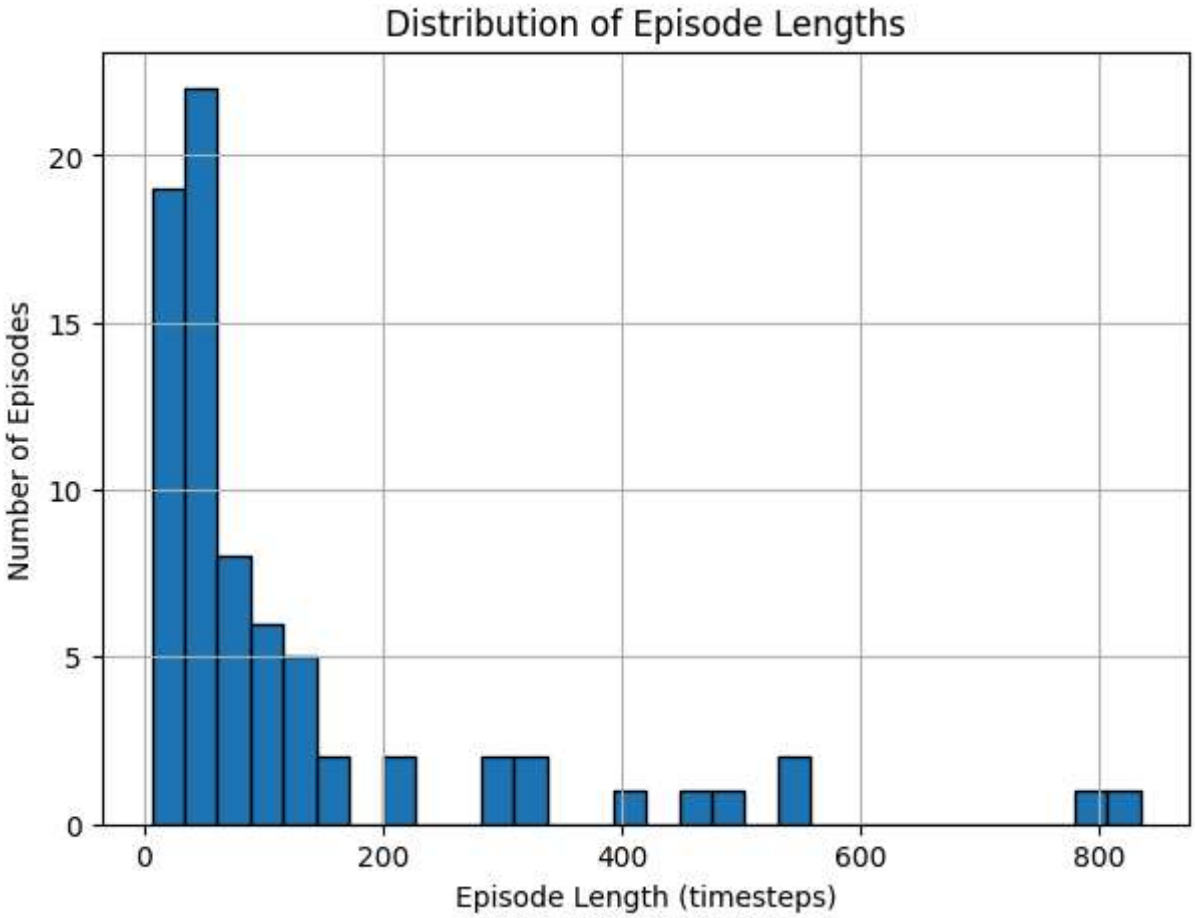
# ----------------- Plot Training Losses -----------------
plt.figure(figsize=(7,5))
plt.plot(actor_losses, label="Actor Loss", marker='o')
plt.plot(critic_losses, label="Critic Loss", marker='s')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Actor & Critic Losses over Training")
plt.legend()
plt.grid(True)
plt.show()

# ----------------- Plot Episode Length Distribution -----------------
episode_lengths = [len(rows) for _, rows in df.groupby(['subject_id','hadm_id','icu

plt.figure(figsize=(7,5))
plt.hist(episode_lengths, bins=30, edgecolor='k')
plt.xlabel("Episode Length (timesteps)")
plt.ylabel("Number of Episodes")
plt.title("Distribution of Episode Lengths")
plt.grid(True)
plt.show()
```


Average Episode Reward over Training
Avg Reward
Average Reward per Transition
−540
−550
−560
−570
−580
−590
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Epoch


Actor & Critic Losses over Training
1e6
Actor Loss
Critic Loss
Loss
7
6
5
4
3
2
1
0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Epoch



Provide a 200-word writeup on the behavior, reward trends, and stability of the trained policy (0.5 Mark)

In our group project, we implemented an Actor–Critic approach to the sepsis treatment problem using the provided ICU dataset. We represented each patient's state using their vitals (mean blood pressure, SpO_2, and respiratory rate) along with demographic features (age and gender). The action space consisted of 99 treatments, including a "NO_ACTION" choice. For the reward function, we used squared error penalties, so states closer to normal ranges (90 mmHg BP, 98% SpO_2, and 16 breaths/minute) gave higher rewards.

During training, we observed that the average episode reward curve stayed almost flat. At first, we thought this was a mistake, but later we realized it was expected because we worked with offline data. Since the next states and rewards are already logged in the dataset, the agent's chosen actions cannot actually change them, so the overall reward trend does not improve over epochs.

We also checked the loss curves. The actor loss stayed fairly stable, suggesting the policy was converging to some distribution of actions. The critic loss, on the other hand, increased and was less stable, which shows the difficulty of estimating values from offline data. Overall, our trained policy remained stable, but the reward trend did not show much improvement.